192425037

EXP40:

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

#include <stdlib.h>

#include <math.h>


#define MAXC 60


/* letters in decreasing order of English frequency */

const char *ORDER = "ETAOINSHRDLCUMWFGYPBVKJXQZ";

const char *BIG[] =
{"TH","HE","IN","ER","AN","RE","ON","AT","EN","ND","TI","ES","OR","TE","OF",


"ED","IS","IT","AL","AR","ST","TO","NT","NG","SE","HA","AS","OU","IO","LE","VE","
CO","ME",


"DE","HI","RI","RO","IC","NE","EA","RA","CE","LI","CH","LL","BE","MA","SI","OM","
UR",0};

const char *QUAD[] =
{"THAT","THER","WITH","TION","HERE","OULD","IGHT","HAVE","THIS","ATIO","F
ROM",


"THEY","EVER","NTHE","ETHE","OFTH","INTH","ANDT","TTHE","ENTS","MENT","
OTHE","WHIC","HICH","THEM",


"THEN","WERE","BEEN","ERED","ANDS","IONS","NING","EACH","OTHER","ALLY",
"ABLE","TING","SAND","IVED",0};

const char *TRI[] =
{"THE","AND","ING","ENT","ION","HER","FOR","THA","NTH","INT","ERE","TIO",


"TER","EST","ERS","ATI","HAT","ATE","ALL","ETH","HES","VER","HIS","OFT","ITH"
,"FTH","STH",
```

```
"OTH","RES","ONT","WIT","ITS","ARE","NCE","EDT","TED","OUR","ONS","IVE","O
ME",0};


double freqE[26] = {8.17,1.49,2.78,4.25,12.70,2.23,2.02,6.09,6.97,0.15,0.77,4.03,2.41,
                    6.75,7.51,1.93,0.10,5.99,6.33,9.06,2.76,0.98,2.36,0.15,1.97,0.07};
double bi[26][26], tri[26][26][26], quad[26][26][26][26], uni[26];
char ct[2000];
int len;


void init_tables() {
   int i, a, b, c;
   for (a = 0; a < 26; a++) for (b = 0; b < 26; b++) bi[a][b] = -1.0;
   for (i = 0; BIG[i]; i++) bi[BIG[i][0] - 'A'][BIG[i][1] - 'A'] = 3.0 - 2.0 * i / 50.0;
   for (i = 0; TRI[i]; i++) tri[TRI[i][0] - 'A'][TRI[i][1] - 'A'][TRI[i][2] - 'A'] = 5.0 - 3.0 * i /
40.0;
   for (i = 0; QUAD[i]; i++)
      quad[QUAD[i][0]-'A'][QUAD[i][1]-'A'][QUAD[i][2]-'A'][QUAD[i][3]-'A'] = 8.0 - 3.0 *
i / 40.0;
   for (i = 0; i < 26; i++) uni[i] = 0.6 * log(freqE[i]);
}


double score(const char *key) {          /* key[c] = plaintext letter for cipher letter c */
   int i; double s = 0;
   for (i = 0; i + 1 < len; i++) {
      int a = key[ct[i] - 'A'] - 'A', b = key[ct[i + 1] - 'A'] - 'A';
      s += uni[a] + bi[a][b];
      if (i + 2 < len) s += tri[a][b][key[ct[i + 2] - 'A'] - 'A'];
      if (i + 3 < len) s += quad[a][b][key[ct[i + 2] - 'A'] - 'A'][key[ct[i + 3] - 'A'] - 'A'];
   }
   return s;
}
```

```
typedef struct { char key[27]; double sc; } Res;
Res res[MAXC];
int nres = 0;
unsigned seed = 12345;
int rnd(int n) { seed = seed * 1103515245u + 12345u; return (seed >> 16) % n; }

int main() {
    int freq[26] = {0}, idx[26], i, j, top, r, improved;
    char key[27], best[27];

    printf("Enter ciphertext: ");
    scanf("%s", ct);
    printf("How many top plaintexts do you want? ");
    scanf("%d", &top);
    for (i = 0; ct[i]; i++) ct[i] = toupper(ct[i]);
    len = strlen(ct);
    init_tables();

    /* initial key: i-th most frequent cipher letter -> i-th most frequent English letter */
    for (i = 0; i < len; i++) freq[ct[i] - 'A']++;
    for (i = 0; i < 26; i++) idx[i] = i;
    for (i = 0; i < 25; i++) for (j = 0; j < 25 - i; j++)
        if (freq[idx[j]] < freq[idx[j + 1]]) { int t = idx[j]; idx[j] = idx[j + 1]; idx[j + 1] = t; }
    for (i = 0; i < 26; i++) best[idx[i]] = ORDER[i];
    best[26] = '\0';

    for (r = 0; r < MAXC; r++) {
        double cur;
        strcpy(key, best);
```

```
    for (i = 0; i < r + 4 * (r > 0); i++) {            /* random restarts */
        int a = rnd(26), b = rnd(26); char t = key[a]; key[a] = key[b]; key[b] = t;
    }
    cur = score(key);
    do {                                /* hill climbing by swapping letters */
        improved = 0;
        for (i = 0; i < 26; i++) for (j = i + 1; j < 26; j++) {
            char t = key[i]; double s;
            key[i] = key[j]; key[j] = t;
            s = score(key);
            if (s > cur + 1e-9) { cur = s; improved = 1; }
            else { t = key[i]; key[i] = key[j]; key[j] = t; }
        }
    } while (improved);
    for (i = 0; i < nres; i++) if (strcmp(res[i].key, key) == 0) break;
    if (i == nres) { strcpy(res[nres].key, key); res[nres].sc = cur; nres++; }
}

for (i = 0; i < nres - 1; i++) for (j = 0; j < nres - 1 - i; j++)
    if (res[j].sc < res[j + 1].sc) { Res t = res[j]; res[j] = res[j + 1]; res[j + 1] = t; }

if (top > nres) top = nres;
printf("\nTop %d candidate plaintexts (most likely first):\n", top);
for (r = 0; r < top; r++) {
    printf("%2d) score %.1f  key(A-Z)=%s  ", r + 1, res[r].sc, res[r].key);
    for (i = 0; i < len && i < 30; i++) putchar(tolower(res[r].key[ct[i] - 'A']));
    printf("...\n");
}
printf("\nBest guess (full text):\n");
for (i = 0; i < len; i++) putchar(tolower(res[0].key[ct[i] - 'A']));
```

```
    printf("\n");
    return 0;
}
```

OUTPUT:

```
Enter ciphertext: XIRRVMNZVUDNRGDUNARQWMUFZQONJMKRZHVZDVUQMNXDVNKXNUSUQJMJMNUPKXMAUXWDUMMUMXDVN
KXNXNNXFWUQMAOHHXHAXJMHRRWGRQNKUUXMOUMNAXJODNKUUDIODUUQPZMNNKUQUGRQUNKODWXBRZNNKUAKRHUVUMOIDXDV
DRNTZMNXBRZNRDUEXQNRGONNKOMPUXDMFKUFWODINKUEXMMARQVMNKUWUJMNKUEQRNRFRHMXDVNKUEUREHUAKRZMUNKUPBU
FXZMUXFKXODOMRDHJXMMNQRDIXMONMAUXWUMNHODWXDVNKUAUXWUMNHODWOMRGNUDMRPUNKODINKXNDRBRVJKXMNKRZIKNX
BRZNBUGRQU
How many top plaintexts do you want? 3

Top 3 candidate plaintexts (most likely first):
 1) score 973.7  key(A-Z)=MCZNYBFWGPHQSTIKROVJEDLAXU  agoodstudentofnetmorlseburitps...
 2) score 971.1  key(A-Z)=CBQNYPFLGWHXSTIKROVJEDMAZU  agoodstudentofnetcormsepuritws...
 3) score 965.6  key(A-Z)=MCZNYBWUGPHQSTIKROVJEDLAXF  agoodstfdentownetmorlsebfritps...

Best guess (full text):
agoodstudentofnetmorlseburitpshouwdunderstandthateverpspstekhasmealnessesandthatattablersmiwwaw
mapswoolfortheeasiestmapintheengineerkustthereforethinlacoutthemhowedesignandnotjustacoutoneyar
tofitthiskeansbheblingtheyassmordsthelepstheyrotobowsandtheyeoywemhousethekcebauseabhainisonwpa
sstrongasitsmealestwinlandthemealestwinlisoftensokethingthatnocodphasthoughtacoutcefore
```